

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

KRITARTH TELI (1BF24CS150)

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum) Department
of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by **KRITARTH TELI (1BF24CS150)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

**Dr. Namratha M
Assistant Professor
Department of CSE
BMSCE, Bengaluru**

**Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru**

Index Sheet

Sl. No.	Experiment Title	Page No.
1	Write program to obtain the Topological ordering of vertices in a given digraph. LeetCode Program related to Topological sorting.	4 - 7
2	Implement Johnson Trotter algorithm to generate permutations.	8 - 10
3	Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort. LeetCode Program related to sorting.	11 - 14
4	Sort a given set of N integer elements using Quick Sort technique and compute its time taken. LeetCode Program related to sorting.	15 - 18
5	Sort a given set of N integer elements using Heap Sort technique and compute its time taken.	19 - 21
6	Implement 0/1 Knapsack problem using dynamic programming. LeetCode Program related to Knapsack problem or Dynamic Programming.	22 - 24
7	Implement All Pair Shortest paths problem using Floyd's algorithm. LeetCode Program related to shortest distance calculation.	25 - 28
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	29 - 32
9	Implement Fractional Knapsack using Greedy technique. LeetCode Program related to Greedy Technique algorithms.	33 - 36
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	37 - 38
11	Implement "N-Queens Problem" using Backtracking.	39 - 40

Lab program 1:

Write program to obtain the Topological ordering of vertices in a given digraph.

Code:

```
#include <stdio.h>
#define MAX 100

int graph[MAX][MAX];
int visited[MAX];
int stack[MAX];
int top = -1;
int n;

void DFS(int v) {
    visited[v] = 1;

    for(int i = 0; i < n; i++)
        if(graph[v][i] == 1 && !visited[i])
            DFS(i);

    stack[++top] = v;
}

void topologicalSort() {
    for(int i = 0; i < n; i++)
        if(!visited[i])
            DFS(i);

    printf("\nTopological Ordering:\n");

    while(top != -1)
        printf("%d ", stack[top--]);
}

int main() {
    int edges, u, v;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter number of edges: ");
    scanf("%d", &edges);
```

```

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            graph[i][j] = 0;

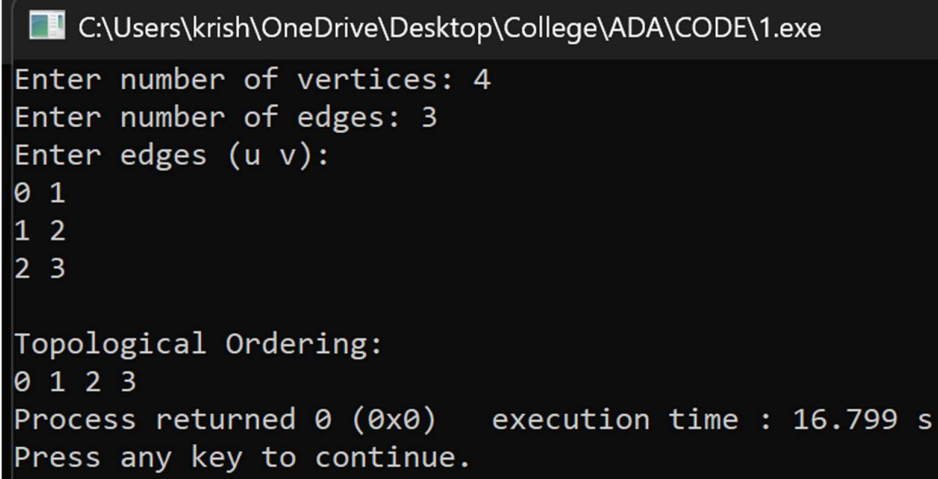
    printf("Enter edges (u v):\n");

    for(int i = 0; i < edges; i++) {
        scanf("%d %d", &u, &v);
        graph[u][v] = 1;
    }

    topologicalSort();
    return 0;
}

```

OUTPUT:



C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\1.exe

```

Enter number of vertices: 4
Enter number of edges: 3
Enter edges (u v):
0 1
1 2
2 3

Topological Ordering:
0 1 2 3
Process returned 0 (0x0)   execution time : 16.799 s
Press any key to continue.

```

LeetCode 207 - Course Schedule

```
bool canFinish(int numCourses, int** prerequisites, int prerequisitesSize,
               int* prerequisitesColSize) {
    int graph[2000][2000] = {0};
    int indegree[2000] = {0};
    int size[2000] = {0};

    for(int i = 0; i < prerequisitesSize; i++) {
        int a = prerequisites[i][0];
        int b = prerequisites[i][1];
        graph[b][size[b]++] = a;
        indegree[a]++;
    }
    int queue[2000];
    int front = 0, rear = 0;

    for(int i = 0; i < numCourses; i++)
        if(indegree[i] == 0)
            queue[rear++] = i;
    int count = 0;

    while(front < rear) {
        int node = queue[front++];
        count++;

        for(int i = 0; i < size[node]; i++) {
            int next = graph[node][i];
            indegree[next]--;
            if(indegree[next] == 0)
                queue[rear++] = next;
        }
    }

    return count == numCourses;
}
```

OUTPUT:

Accepted

Runtime: 4 ms

✓ Case 1

✓ Case 2

Input

numCourses =

2

prerequisites =

[[1,0]]

Output

true

Expected

true

Lab program 2:

Implement Johnson Trotter algorithm to generate permutations.

Code:

```
#include <stdio.h>
#define LEFT -1
#define RIGHT 1

void printPermutation(int a[], int n) {
    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);
    printf("\n");
}

int getMobile(int a[], int dir[], int n) {
    int mobile = 0, pos = -1;

    for(int i = 0; i < n; i++) {
        if(dir[i] == LEFT && i != 0 && a[i] > a[i - 1]) {
            if(a[i] > mobile) {
                mobile = a[i];
                pos = i;
            }
        }

        if(dir[i] == RIGHT && i != n - 1 && a[i] > a[i + 1]) {
            if(a[i] > mobile) {
                mobile = a[i];
                pos = i;
            }
        }
    }

    return pos;
}

void johnsonTrotter(int n) {
    int a[n], dir[n];

    for(int i = 0; i < n; i++) {
        a[i] = i + 1;
        dir[i] = LEFT;
    }
}
```

```

printPermutation(a, n);

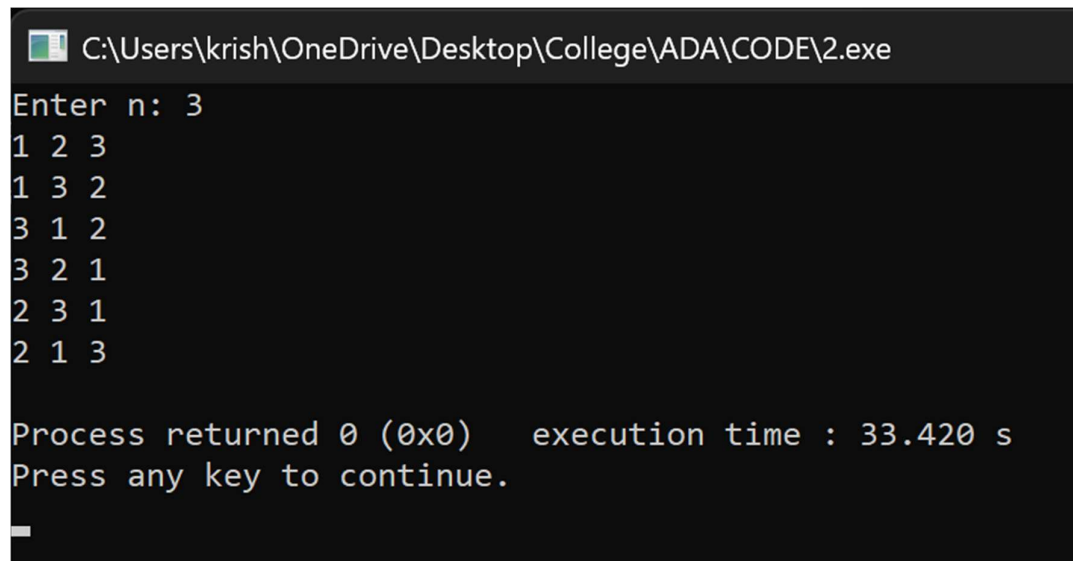
while(1) {
    int pos = getMobile(a, dir, n);
    if(pos == -1)
        break;
    int swapPos;
    if(dir[pos] == LEFT)
        swapPos = pos - 1;
    else
        swapPos = pos + 1;
    int temp = a[pos];
    a[pos] = a[swapPos];
    a[swapPos] = temp;
    temp = dir[pos];
    dir[pos] = dir[swapPos];
    dir[swapPos] = temp;
    pos = swapPos;
    for(int i = 0; i < n; i++)
        if(a[i] > a[pos])
            dir[i] = -dir[i];
    printPermutation(a, n);
}
}

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);
    johnsonTrotter(n);

    return 0;
}

```

OUTPUT:



```
C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\2.exe
Enter n: 3
1 2 3
1 3 2
3 1 2
3 2 1
2 3 1
2 1 3

Process returned 0 (0x0)   execution time : 33.420 s
Press any key to continue.
```

Lab program 3:

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void merge(int a[], int low, int mid, int high) {
    int i = low, j = mid + 1, k = 0;
    int temp[100000];

    while(i <= mid && j <= high) {
        if(a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while(i <= mid)
        temp[k++] = a[i++];

    while(j <= high)
        temp[k++] = a[j++];

    for(i = low, k = 0; i <= high; i++, k++)
        a[i] = temp[k];
}

void mergeSort(int a[], int low, int high) {
    if(low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
```

```
int main() {
    int n;
    int a[100000];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        a[i] = rand() % 1000;
    clock_t start, end;
    start = clock();
    mergeSort(a, 0, n - 1);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nSorted Elements:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nTime Taken = %f seconds\n", time_taken);

    return 0;
}
```

OUTPUT:

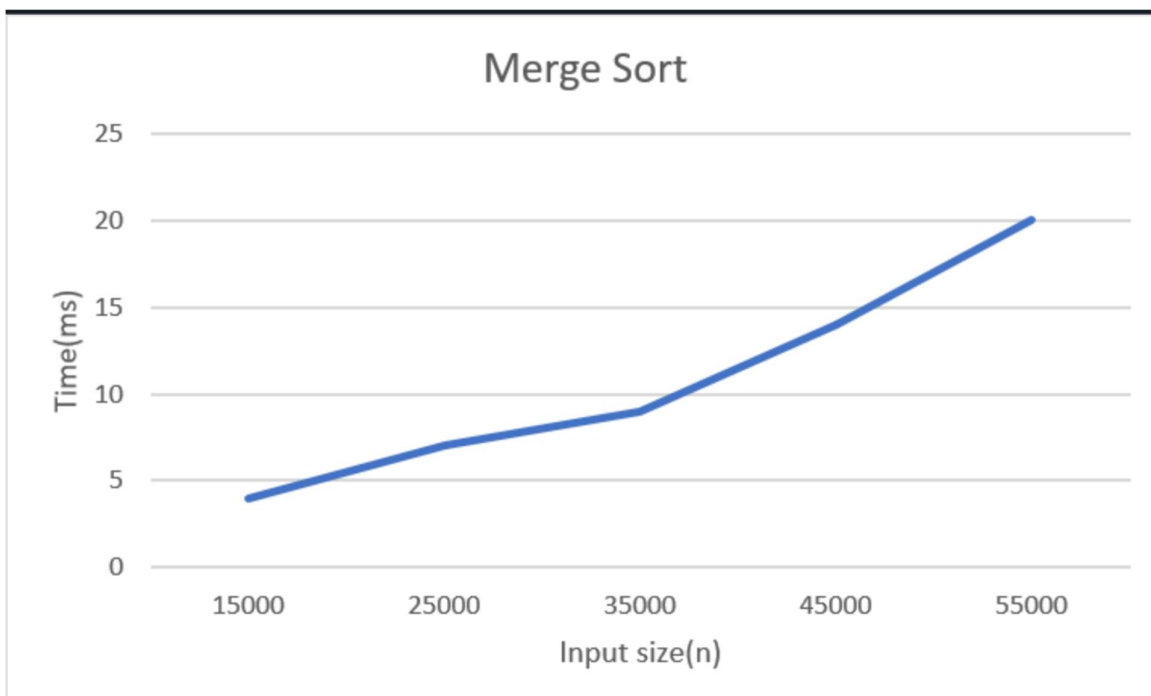
```
C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\3.exe
Enter number of elements: 10

Sorted Elements:
41 169 334 358 464 467 478 500 724 962

Time Taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 32.283 s
Press any key to continue.
■
```

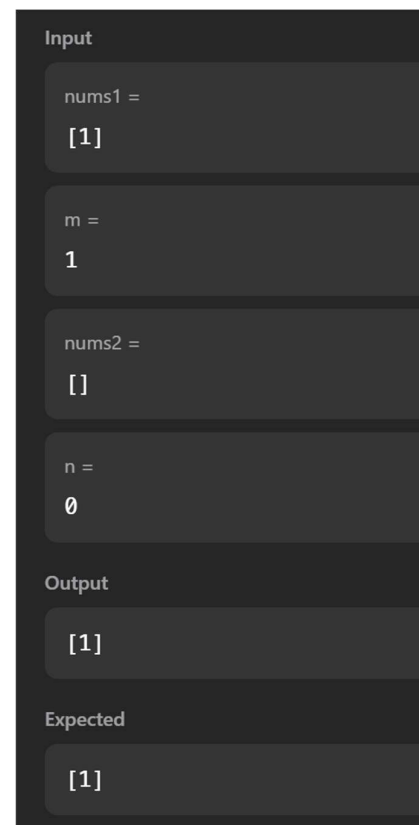
GRAPH:



LeetCode 88 - Merge Sorted Array

```
void merge(int* nums1, int m, int* nums2, int n) {  
    int i = m - 1;  
    int j = n - 1;  
    int k = m + n - 1;  
  
    while(i >= 0 && j >= 0) {  
        if(nums1[i] > nums2[j])  
            nums1[k--] = nums1[i--];  
        else  
            nums1[k--] = nums2[j--];  
    }  
  
    while(j >= 0)  
        nums1[k--] = nums2[j--];  
}
```

OUTPUT:



The screenshot shows a dark-themed interface with the following sections:

- Input**
 - nums1 = [1]
 - m = 1
 - nums2 = []
 - n = 0
- Output**
 - [1]
- Expected**
 - [1]

Lab program 4:

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int a[], int low, int high) {
    int pivot = a[low];
    int i = low + 1;
    int j = high;

    while(i <= j) {
        while(i <= high && a[i] <= pivot)
            i++;
        while(a[j] > pivot)
            j--;
        if(i < j)
            swap(&a[i], &a[j]);
    }

    swap(&a[low], &a[j]);
    return j;
}

void quickSort(int a[], int low, int high) {
    if(low < high) {
        int p = partition(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

```
int main() {
    int n;
    int a[100000];
    printf("Enter number of elements: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        a[i] = rand() % 1000;

    clock_t start, end;
    start = clock();
    quickSort(a, 0, n - 1);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nSorted Elements:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nTime Taken = %f seconds\n", time_taken);

    return 0;
}
```

OUTPUT:

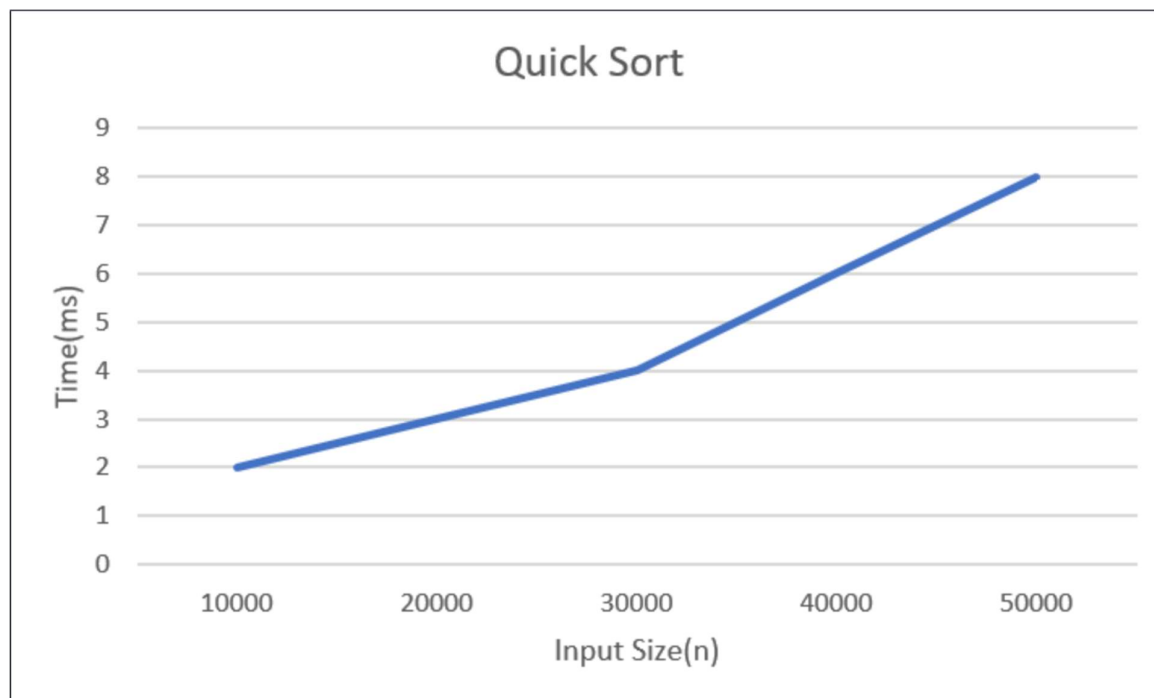
```
C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\4.exe
Enter number of elements: 10

Sorted Elements:
41 169 334 358 464 467 478 500 724 962

Time Taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 2.523 s
Press any key to continue.
```

GRAPH:

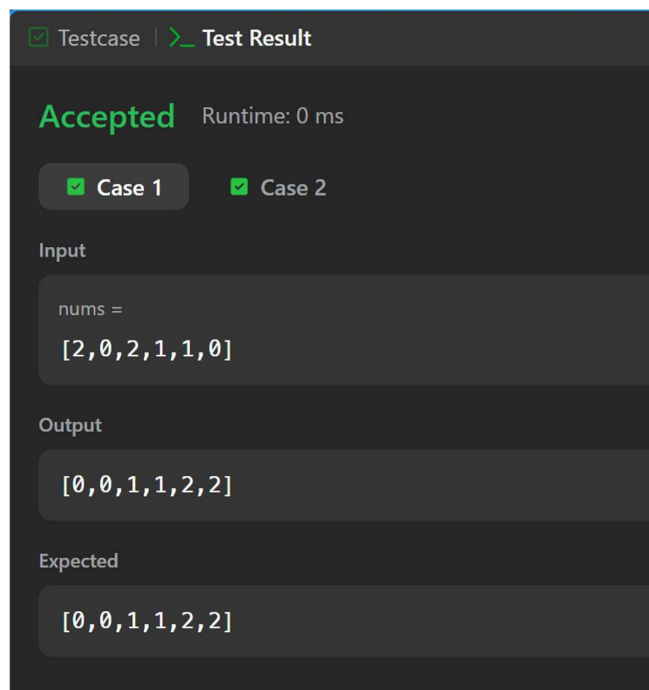


LeetCode 75 - Sort Colors

```
void sortColors(int* nums, int numsSize) {
    int low = 0, mid = 0, high = numsSize - 1;

    while(mid <= high) {
        if(nums[mid] == 0) {
            int temp = nums[low];
            nums[low] = nums[mid];
            nums[mid] = temp;
            low++;
            mid++;
        }
        else if(nums[mid] == 1)
            mid++;
        else {
            int temp = nums[mid];
            nums[mid] = nums[high];
            nums[high] = temp;
            high--;
        }
    }
}
```

OUTPUT:



Lab program 5:

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

void heapify(int a[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if(left < n && a[left] > a[largest])
        largest = left;

    if(right < n && a[right] > a[largest])
        largest = right;

    if(largest != i) {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest);
    }
}

void heapSort(int a[], int n) {
    for(int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);

    for(int i = n - 1; i > 0; i--) {
        swap(&a[0], &a[i]);
        heapify(a, i, 0);
    }
}
```

```
int main() {
    int n;
    int a[100000];
    printf("Enter number of elements: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        a[i] = rand() % 1000;

    clock_t start, end;
    start = clock();
    heapSort(a, n);
    end = clock();
    double time_taken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("\nSorted Elements:\n");

    for(int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n\nTime Taken = %f seconds\n", time_taken);

    return 0;
}
```

OUTPUT:

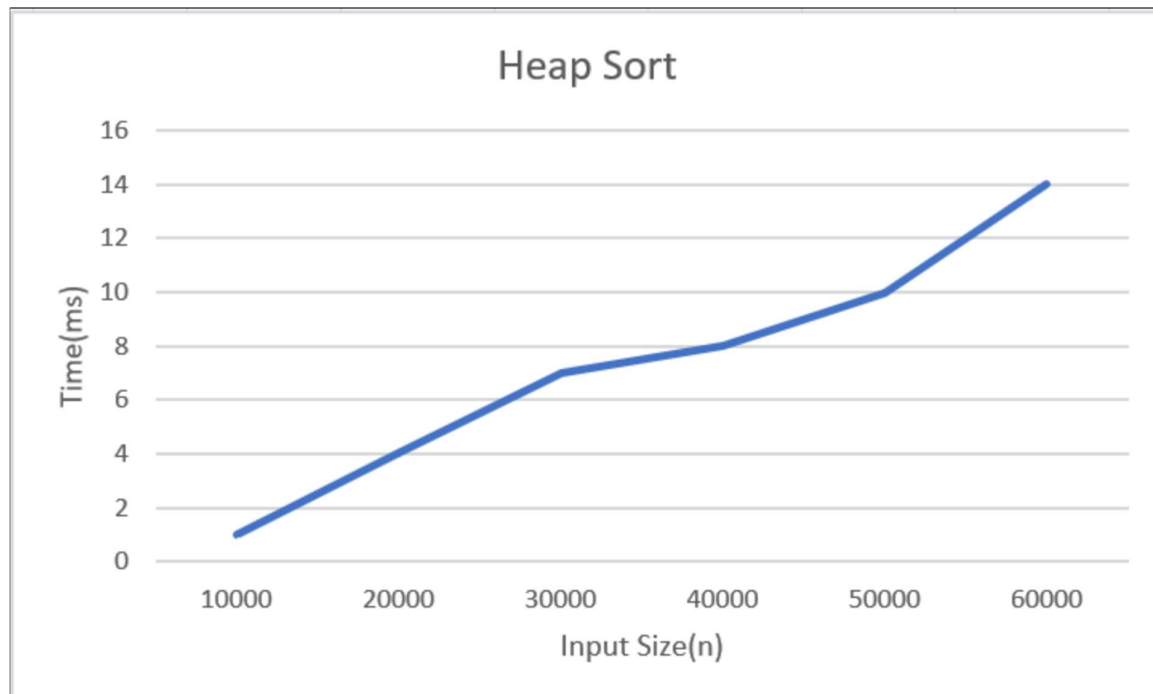
```
C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\5.exe
Enter number of elements: 10

Sorted Elements:
41 169 334 358 464 467 478 500 724 962

Time Taken = 0.000000 seconds

Process returned 0 (0x0)   execution time : 8.672 s
Press any key to continue.
_
```

GRAPH:



Lab program 6:

Implement 0/1 Knapsack problem using dynamic programming.

Code:

```
#include <stdio.h>

int max(int a, int b) {
    if(a > b)
        return a;
    return b;
}

int main() {
    int n, W;
    printf("Enter number of items: ");
    scanf("%d", &n);
    int wt[n], val[n];
    printf("Enter weights:\n");

    for(int i = 0; i < n; i++)
        scanf("%d", &wt[i]);

    printf("Enter profits:\n");

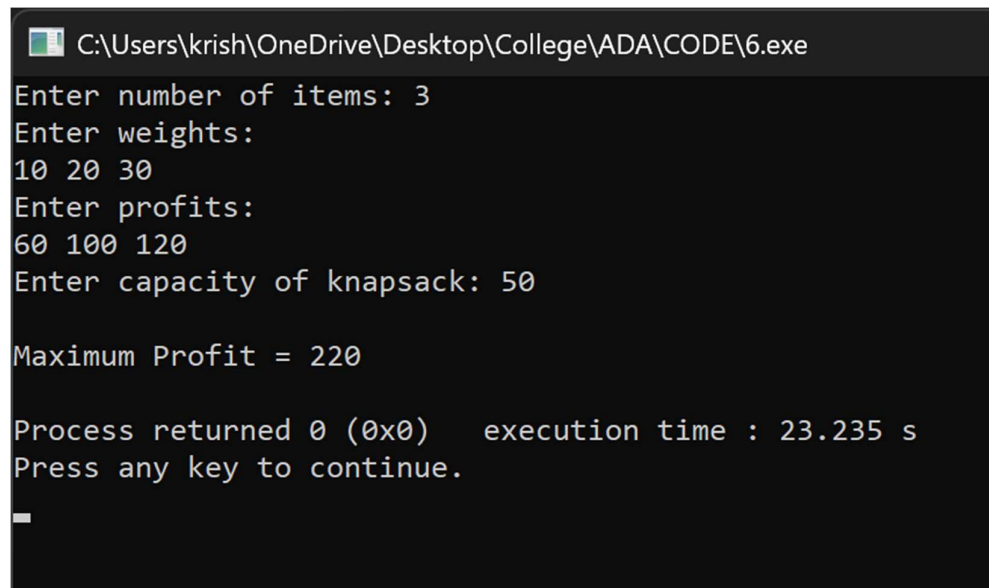
    for(int i = 0; i < n; i++)
        scanf("%d", &val[i]);

    printf("Enter capacity of knapsack: ");
    scanf("%d", &W);
    int dp[n + 1][W + 1];

    for(int i = 0; i <= n; i++) {
        for(int w = 0; w <= W; w++) {
            if(i == 0 || w == 0)
                dp[i][w] = 0;
            else if(wt[i - 1] <= w)
                dp[i][w] = max(val[i - 1] + dp[i - 1][w - wt[i - 1]], dp[i - 1][w]);
            else
                dp[i][w] = dp[i - 1][w];
        }
    }

    printf("\nMaximum Profit = %d\n", dp[n][W]);
    return 0;
}
```

OUTPUT:



```
C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\6.exe
Enter number of items: 3
Enter weights:
10 20 30
Enter profits:
60 100 120
Enter capacity of knapsack: 50

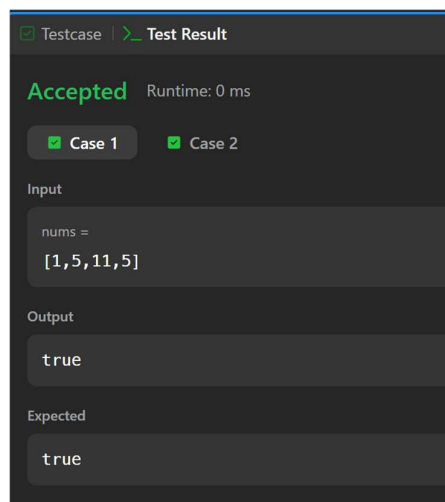
Maximum Profit = 220

Process returned 0 (0x0)   execution time : 23.235 s
Press any key to continue.
■
```

LeetCode 416 - Partition Equal Subset Sum

```
bool canPartition(int* nums, int numsSize) {  
    int sum = 0;  
  
    for(int i = 0; i < numsSize; i++)  
        sum += nums[i];  
  
    if(sum % 2 != 0)  
        return false;  
  
    int target = sum / 2;  
    bool dp[target + 1];  
  
    for(int i = 0; i <= target; i++)  
        dp[i] = false;  
  
    dp[0] = true;  
  
    for(int i = 0; i < numsSize; i++) {  
        for(int j = target; j >= nums[i]; j--) {  
            if(dp[j - nums[i]])  
                dp[j] = true;  
        }  
    }  
  
    return dp[target];  
}
```

OUTPUT:



Lab program 7:

Implement All Pair Shortest paths problem using Floyd's algorithm.

CODE:

```
#include <stdio.h>
#define INF 99999

void floydWarshall(int graph[][10], int V) {
    int dist[10][10];

    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++)
            dist[i][j] = graph[i][j];
    }

    for(int k = 0; k < V; k++) {
        for(int i = 0; i < V; i++) {
            for(int j = 0; j < V; j++) {
                if(dist[i][k] != INF && dist[k][j] != INF && dist[i][k] + dist[k][j] < dist[i][j])
                    dist[i][j] = dist[i][k] + dist[k][j];
            }
        }
    }

    printf("Shortest Distance Matrix:\n");

    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++) {
            if(dist[i][j] == INF)
                printf("INF ");
            else
                printf("%3d ", dist[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int V;
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    int graph[10][10];
    printf("Enter adjacency matrix:\n");
    printf("(Enter %d for INF)\n", INF);
```

```

    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);
    }
    floydWarshall(graph, V);

    return 0;
}

```

OUTPUT:

```

C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\7.exe
Enter number of vertices: 4
Enter adjacency matrix:
(Enter 99999 for INF)
0 5 99999 10
99999 0 3 99999
99999 99999 0 1
99999 99999 99999 0
Shortest Distance Matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

Process returned 0 (0x0)   execution time : 43.684 s
Press any key to continue.
_

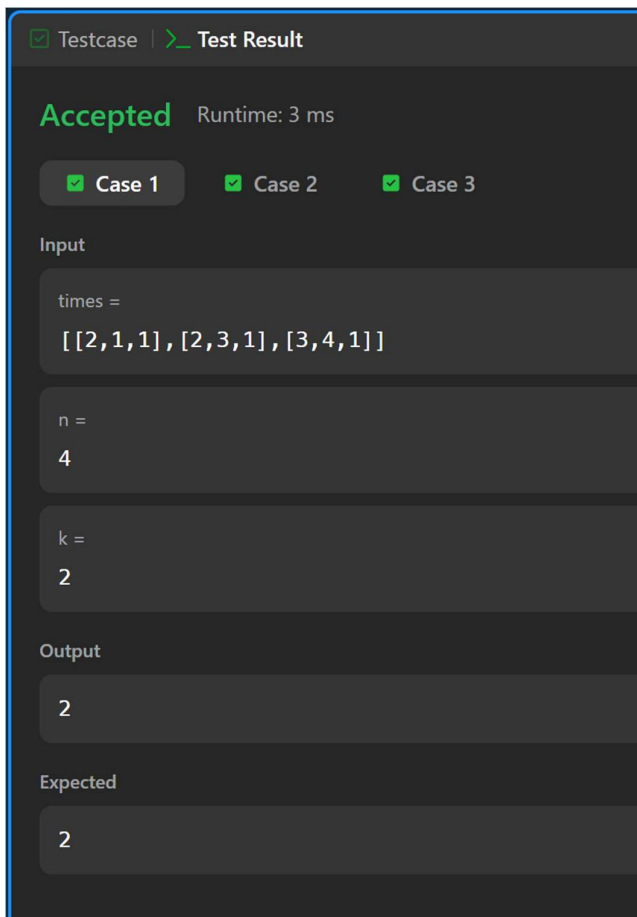
```

LeetCode 743 - Network Delay Time

```
int networkDelayTime(int** times, int timesSize, int* timesColSize, int n, int k) {  
    int dist[101];  
    int visited[101] = {0};  
    for(int i = 1; i <= n; i++)  
        dist[i] = 1000000000;  
    dist[k] = 0;  
    for(int cnt = 0; cnt < n; cnt++) {  
        int u = -1;  
        for(int i = 1; i <= n; i++) {  
            if(!visited[i] && (u == -1 || dist[i] < dist[u]))  
                u = i;  
        }  
        visited[u] = 1;  
        for(int i = 0; i < timesSize; i++) {  
            int from = times[i][0];  
            int to = times[i][1];  
            int w = times[i][2];  
            if(from == u && dist[u] + w < dist[to])  
                dist[to] = dist[u] + w;  
        }  
    }  
    int ans = 0;
```

```
for(int i = 1; i <= n; i++) {  
    if(dist[i] == 1000000000)  
        return -1;  
    if(dist[i] > ans)  
        ans = dist[i];  
}  
return ans;  
}
```

OUTPUT:



Testcase | Test Result

Accepted Runtime: 3 ms

Case 1 Case 2 Case 3

Input

times =
[[2,1,1],[2,3,1],[3,4,1]]

n =
4

k =
2

Output

2

Expected

2

Lab program 8:

a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

CODE:

```
#include <stdio.h>
#define MAX 10
#define INF 99999

int minKey(int key[], int mstSet[], int V) {
    int min = INF, minIndex;

    for(int i = 0; i < V; i++) {
        if(mstSet[i] == 0 && key[i] < min) {
            min = key[i];
            minIndex = i;
        }
    }
    return minIndex;
}

void primMST(int graph[MAX][MAX], int V) {
    int parent[MAX];
    int key[MAX];
    int mstSet[MAX];

    for(int i = 0; i < V; i++) {
        key[i] = INF;
        mstSet[i] = 0;
    }
    key[0] = 0;
    parent[0] = -1;

    for(int count = 0; count < V-1; count++) {
        int u = minKey(key, mstSet, V);
        mstSet[u] = 1;

        for(int v = 0; v < V; v++) {
            if(graph[u][v] && mstSet[v] == 0 && [u][v] < key[v]) {
                parent[v] = u;
                key[v] = graph[u][v];
            }
        }
    }
}
```

```

int cost = 0;
printf("\nEdges in MST:\n");
printf("Edge\tWeight\n");

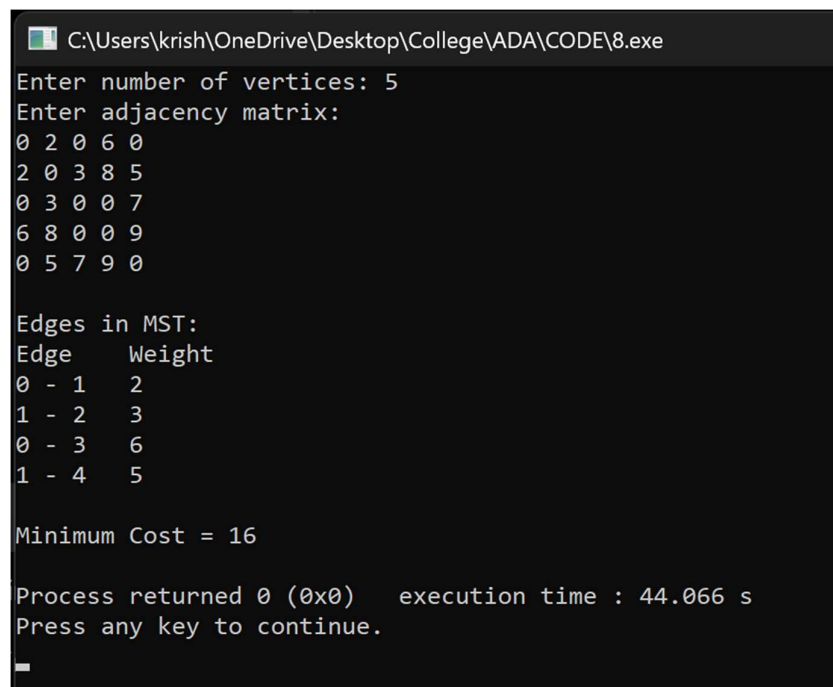
for(int i = 1; i < V; i++) {
    printf("%d - %d\t%d\n", parent[i], i, graph[i][parent[i]]);
    cost += graph[i][parent[i]];
}
printf("\nMinimum Cost = %d\n", cost);
}

int main() {
    int V;
    int graph[MAX][MAX];
    printf("Enter number of vertices: ");
    scanf("%d", &V);
    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);
    }
    primMST(graph, V);
    return 0;
}

```

OUTPUT:



```

C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\8.exe
Enter number of vertices: 5
Enter adjacency matrix:
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges in MST:
Edge    Weight
0 - 1    2
1 - 2    3
0 - 3    6
1 - 4    5

Minimum Cost = 16

Process returned 0 (0x0)   execution time : 44.066 s
Press any key to continue.

```

Lab program 8:

b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

CODE:

```
#include <stdio.h>
#define MAX 20

struct Edge {
    int u, v, w;
};

struct Edge edges[MAX];
int parent[MAX];

int find(int i) {
    while(parent[i] != i)
        i = parent[i];
    return i;
}

void unionSet(int a, int b) {
    parent[a] = b;
}

int main() {
    int V, E;

    printf("Enter number of vertices: ");
    scanf("%d", &V);

    printf("Enter number of edges: ");
    scanf("%d", &E);

    printf("Enter source destination weight:\n");

    for(int i = 0; i < E; i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].w);

    for(int i = 0; i < V; i++)
        parent[i] = i;
```

```

for(int i = 0; i < E - 1; i++) {
    for(int j = 0; j < E - i - 1; j++) {
        if(edges[j].w > edges[j + 1].w) {
            struct Edge temp = edges[j];
            edges[j] = edges[j + 1];
            edges[j + 1] = temp;
        }
    }
}

int count = 0;
int cost = 0;
printf("\nEdges in MST:\n");

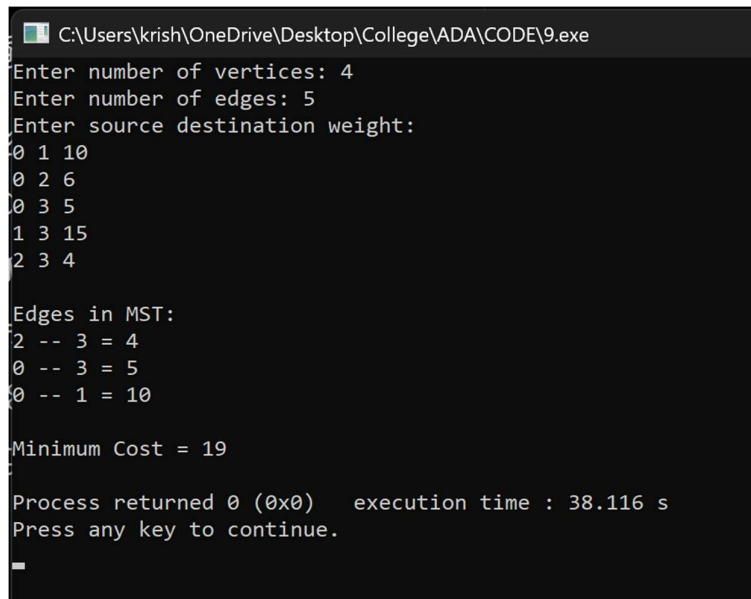
for(int i = 0; i < E && count < V - 1; i++) {
    int a = find(edges[i].u);
    int b = find(edges[i].v);

    if(a != b) {
        printf("%d -- %d = %d\n", edges[i].u, edges[i].v, edges[i].w);
        cost += edges[i].w;
        unionSet(a, b);
        count++;
    }
}

printf("\nMinimum Cost = %d\n", cost);
return 0;
}

```

OUTPUT:



```

C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\9.exe
Enter number of vertices: 4
Enter number of edges: 5
Enter source destination weight:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in MST:
2 -- 3 = 4
0 -- 3 = 5
0 -- 1 = 10

Minimum Cost = 19

Process returned 0 (0x0)   execution time : 38.116 s
Press any key to continue.

```

Lab program 9: Implement Fractional Knapsack using Greedy technique.

CODE:

```
#include <stdio.h>

struct Item {
    int profit;
    int weight;
    float ratio;
};

int main() {
    int n, capacity;
    printf("Enter number of items: ");
    scanf("%d", &n);
    struct Item item[n];
    printf("Enter profit and weight of each item:\n");

    for(int i = 0; i < n; i++) {
        scanf("%d %d", &item[i].profit, &item[i].weight);
        item[i].ratio = (float)item[i].profit / item[i].weight;
    }

    printf("Enter knapsack capacity: ");
    scanf("%d", &capacity);

    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(item[j].ratio < item[j + 1].ratio) {
                struct Item temp = item[j];
                item[j] = item[j + 1];
                item[j + 1] = temp;
            }
        }
    }

    float totalProfit = 0.0;
```

```

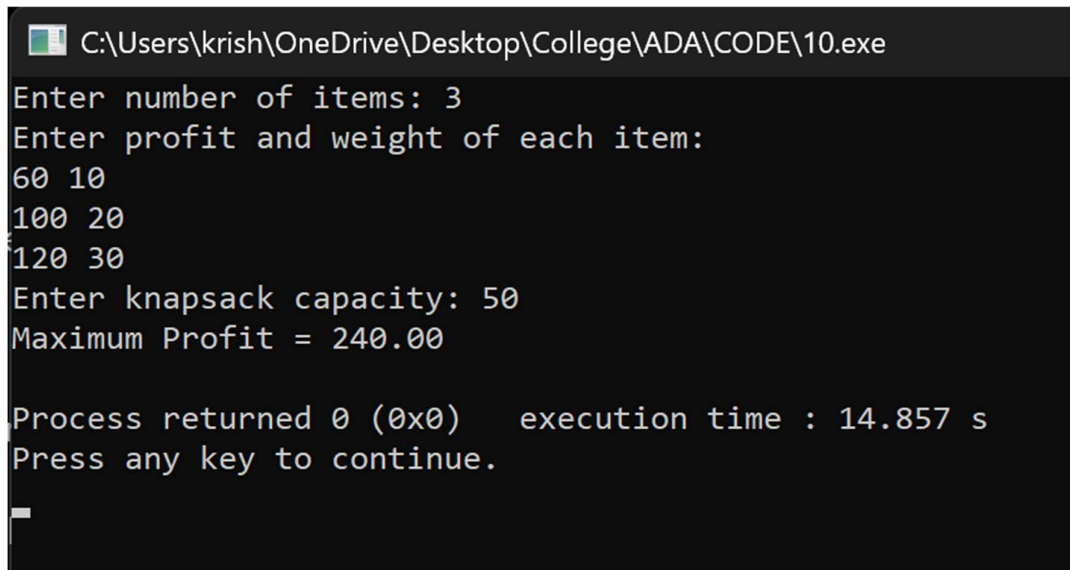
for(int i = 0; i < n; i++) {
    if(capacity >= item[i].weight) {
        totalProfit += item[i].profit;
        capacity -= item[i].weight;
    }
    else {
        totalProfit += item[i].ratio * capacity;
        break;
    }
}

printf("Maximum Profit = %.2f\n", totalProfit);

return 0;
}

```

OUTPUT:



```

C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\10.exe
Enter number of items: 3
Enter profit and weight of each item:
60 10
100 20
120 30
Enter knapsack capacity: 50
Maximum Profit = 240.00

Process returned 0 (0x0)   execution time : 14.857 s
Press any key to continue.

```

LeetCode 455 - Assign Cookies

```
int findContentChildren(int* g, int gSize, int* s, int sSize) {  
    for(int i=0;i<gSize-1;i++) {  
        for(int j=0;j<gSize-i-1;j++) {  
            if(g[j]>g[j+1]){  
                int t=g[j];  
                g[j]=g[j+1];  
                g[j+1]=t;  
            }  
        }  
    }  
    for(int i=0;i<sSize-1;i++) {  
        for(int j=0;j<sSize-i-1;j++) {  
            if(s[j]>s[j+1]) {  
                int t=s[j];  
                s[j]=s[j+1];  
                s[j+1]=t;  
            }  
        }  
    }  
    int i=0,j=0;  
    while(i<gSize && j<sSize) {  
        if(s[j]>=g[i])  
            i++;  
        j++;  
    }  
    return i;  
}
```

OUTPUT:

☒ Testcase |  Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

g =
[1,2,3]

s =
[1,1]

Output

1

Expected

1

Lab program 10:

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

CODE:

```
#include <stdio.h>
#define MAX 10
#define INF 99999

int minDistance(int dist[], int visited[], int V) {
    int min = INF, minIndex;

    for(int i = 0; i < V; i++) {
        if(!visited[i] && dist[i] < min) {
            min = dist[i];
            minIndex = i;
        }
    }
    return minIndex;
}

void dijkstra(int graph[MAX][MAX], int V, int src) {
    int dist[MAX];
    int visited[MAX] = {0};

    for(int i = 0; i < V; i++)
        dist[i] = INF;

    dist[src] = 0;

    for(int count = 0; count < V - 1; count++) {
        int u = minDistance(dist, visited, V);
        visited[u] = 1;

        for(int v = 0; v < V; v++) {
            if(!visited[v] && graph[u][v] && dist[u] != INF && dist[u] + graph[u][v] < dist[v])
                dist[v] = dist[u] + graph[u][v];
        }
    }
    printf("\nVertex\tDistance from Source\n");

    for(int i = 0; i < V; i++)
        printf("%d\t%d\n", i, dist[i]);
}
```

```

int main() {
    int V, src;
    int graph[MAX][MAX];

    printf("Enter number of vertices: ");
    scanf("%d", &V);

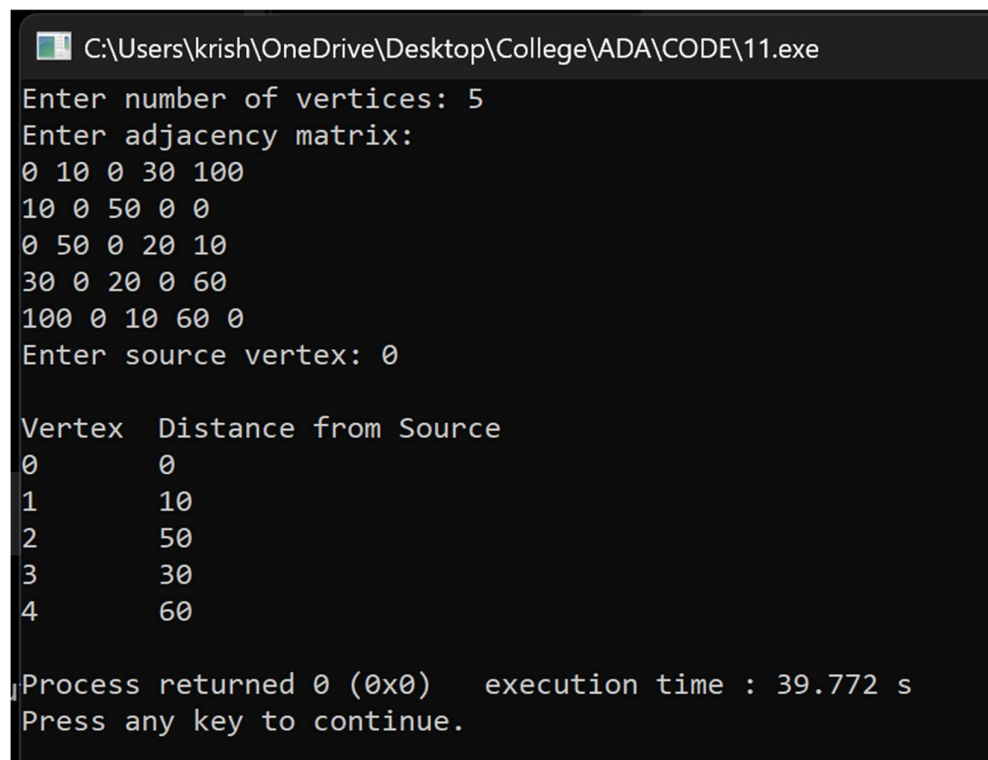
    printf("Enter adjacency matrix:\n");

    for(int i = 0; i < V; i++) {
        for(int j = 0; j < V; j++)
            scanf("%d", &graph[i][j]);
    }
    printf("Enter source vertex: ");
    scanf("%d", &src);
    dijkstra(graph, V, src);

    return 0;
}

```

OUTPUT:



```

C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\11.exe
Enter number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Enter source vertex: 0

Vertex  Distance from Source
0         0
1         10
2         50
3         30
4         60

Process returned 0 (0x0)   execution time : 39.772 s
Press any key to continue.

```

Lab program 11:
Implement “N-Queens Problem” using Backtracking.

CODE:

```
#include <stdio.h>
#define MAX 20
int board[MAX][MAX];

int isSafe(int row, int col, int n) {
    int i, j;

    for(i = 0; i < col; i++)
        if(board[row][i])
            return 0;

    for(i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if(board[i][j])
            return 0;

    for(i = row, j = col; i < n && j >= 0; i++, j--)
        if(board[i][j])
            return 0;

    return 1;
}

int solveNQueens(int col, int n) {
    if(col >= n)
        return 1;

    for(int i = 0; i < n; i++) {
        if(isSafe(i, col, n)) {
            board[i][col] = 1;

            if(solveNQueens(col + 1, n))
                return 1;

            board[i][col] = 0;
        }
    }
    return 0;
}
```

```

int main() {
    int n;
    printf("Enter value of N: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++)
        for(int j = 0; j < n; j++)
            board[i][j] = 0;

    if(solveNQueens(0, n)) {
        printf("\nSolution:\n");

        for(int i = 0; i < n; i++) {
            for(int j = 0; j < n; j++)
                printf("%d ", board[i][j]);

            printf("\n");
        }
    }
    else
        printf("No solution exists\n");

    return 0;
}

```

OUTPUT:

```

C:\Users\krish\OneDrive\Desktop\College\ADA\CODE\12.exe
Enter value of N: 4

Solution 1:
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0

Solution 2:
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0

Total Solutions = 2

Process returned 0 (0x0)   execution time : 1.510 s
Press any key to continue.

```